TECHNICAL CHALLENGE · AI ENGINEER · ARTEFACT

Empório da Música

Customer support agent — technical documentation

Author: Thiago Afonso

Repository: github.com/Thafonso/desafio_artefact

Generated on: November 03, 2026

Contents

Empório da Música — customer support agent

- Running it
- Architecture
  - The tools
  - Language convention
- Technical decisions
  - Hybrid: function calling on the data, retrieval on the policies
  - Why LangGraph
  - Model: claude-haiku-5-5
  - Retrieval: BM25 over the sections, not embeddings
  - PDF extraction
  - Prompt: identity in the prompt, facts in the tools
- Assumptions made
- Known limitations
  - With more time
- Example interactions
- Use of coding assistants

Appendix — example conversations

- Catálogo com filtro de preço
- Informações da loja e pedido fora do catálogo
- Promoção expirada (cenário não trivial)
- Devolução com cálculo de prazo (cenário não trivial)
- Pergunta fora de escopo e rastreio de pedido

Empório da Música — customer support agent

Prototype of a text-message support agent for Empório da Música, a fictional musical instrument store in Campo Grande, Brazil. The agent answers questions about the catalog, prices, orders and store policies, and knows which source to consult for each.

Technical challenge for the AI Engineer role at Artefact.

> The agent talks to Brazilian customers, so everything it says — and everything the CLI prints — is in Portuguese. The code and this document are in English; see Language convention.

Running it

Requirements: Python 3.11+ (developed and tested on 3.13) and an Anthropic API key — console.anthropic.com.

```
git clone https://github.com/Thafonso/desafio_artefact.git
cd desafio_artefact
python -m venv .venv
```

Activate the environment — `.venv\Scripts\activate` on Windows, `source .venv/bin/activate` on Linux/macOS — then:

```
pip install -r requirements.txt
```

Copy `.env.example` to `.env` and fill in the key:

```
cp .env.example .env
```

```
ANTHROPIC_API_KEY=sk-ant-...
```

Talk to the agent:

```
python -m emporio.cli
```

Other modes:

```
python -m emporio.cli -p "quais violões vocês têm até R$1000?"
```

```
python -m emporio.cli -s joao
```

```
python -m emporio.cli --listar
```

`-p` asks one question and exits. `-s` names the session: running again with the same name resumes the conversation where it stopped, even after closing the terminal. `--listar` shows the saved sessions.

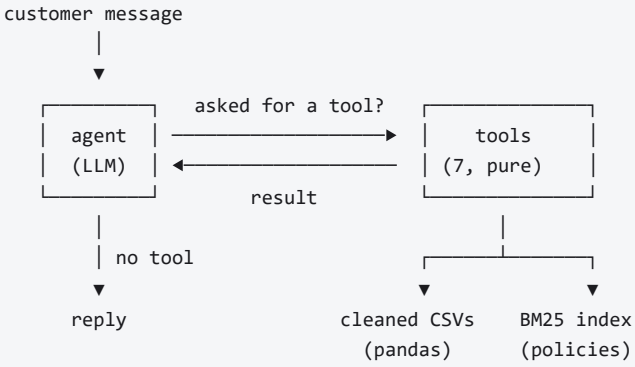
Run the tests:

```
python -m pytest tests -q
```

Regenerate the example conversations (spends API credit):

```
python scripts/generate_examples.py
```

Architecture



File	Role
`emporio/config.py`	Keys, paths and the reference date
`emporio/data.py`	CSVs → normalized, queryable DataFrames
`emporio/policies.py`	PDF → 29 sections → BM25 index
`emporio/tools.py`	The 7 tools the model can call
`emporio/prompts.py`	Persona, routing and limits
`emporio/agent.py`	The LangGraph graph
`emporio/cli.py`	Conversation interface

The tools

Tool	What for
`search_products`	Catalog, filtered by term, category and price range
`product_details`	Full record: specs, stock, prices
`list_categories`	What the store sells and how many of each
`active_promotions`	Live promotions only
`lookup_order`	Status, items, tracking and exchange windows
`payment_options`	PIX, valid instalments and shipping
`search_policies`	Searches the manual, returns a quotable passage

Language convention

The code is in English — module, function and variable names, comments and docstrings. Portuguese is reserved for **content**, not structure: the system prompt (it defines a persona talking to Brazilian customers), the text the tools return, the CLI messages and the flags the store's team would type. The line is simple: if a person reads it on screen, it is Portuguese; if it is the program talking to the program, it is English.

The exception that matters: the **tool names and descriptions are in English**, and they are not just documentation — they travel in the API payload and are what the model reads to pick one. In practice Claude routes just as well with English descriptions and a Portuguese conversation, but it is a change that affects behaviour, not only readability.

Technical decisions

Hybrid: function calling on the data, retrieval on the policies

The store's data is tabular and small (65 products, 20 orders). Real questions about it are filters and aggregations — "*violões até R$1000*", "*cadê meu pedido 8*" — and that is exactly where similarity search fails: an embedding does not know that 999 ≤ 1000. So the catalog is exposed as **deterministic tools** over DataFrames.

I considered a **SQL agent**, which would give more query flexibility. For 65 rows it does not pay off: it adds an entire failure surface (invalid SQL, injection, silently wrong joins) without solving anything the seven tools do not already solve. If the catalog were the 300k rows a real store would have, the trade would invert.

The **policies** are the opposite case: prose in natural language, where the customer's question almost never uses the manual's words. There, retrieval is the right tool.

Section 7.1 of the manual settles it: "*never give information about prices, stock or deadlines without checking the system*". The agent is built so that this is structurally true, not an instruction the model may ignore.

Why LangGraph

The `model → tool → model` loop is ~40 hand-written lines, and I did consider writing it that way to avoid the dependency. I kept LangGraph for one concrete reason: the **checkpointer**. It persists graph state by `thread_id`, which gives resumable conversations with the tool calls included, without me writing a serialization layer. The CLI uses `SqliteSaver` and the example generator uses `InMemorySaver` — the difference between them is one argument to `build_graph`.

The graph is written out explicitly (`agent` → `tools_condition` → `tools` → `agent`) instead of `create_react_agent`, because the loop is the heart of the project and hiding it behind one call would remove the most interesting thing to read.

Model: `claude-haiku-5-5`

The task is routing between seven well-described tools and writing short Portuguese replies — it does not need heavy reasoning. Haiku 5.5 is the cheapest model of the current generation and its function calling is reliable; in a support channel running thousands of conversations, that is the difference between viable and not. The model is configurable through `EMPORIO_MODEL`, so moving up to Sonnet is one line in `.env`.

One quirk: Haiku 5.5 rejects sampling values other than the default (`temperature`, `top_p`, `top_k`). I inspected the payload assembled by `langchain-anthropic` before making any call, to confirm it does not send those fields.

Retrieval: BM25 over the sections, not embeddings

The manual is 8 pages. It would fit whole in the system prompt, and that would work. I chose to expose it as a **tool** for two reasons: it keeps the prompt prefix small and stable, and it makes the decision "*this is a policy question*" explicit and auditable — which is precisely the behaviour the challenge asks the agent to get right. In the generated examples you can watch the agent choosing between querying data and querying the manual.

The search is lexical (BM25) rather than vector-based, because with ~25 short passages and Portuguese questions full of the manual's literal terms (*troca, garantia, boleto, frete*) it hits, and it avoids pulling in a second provider just to generate embeddings.

A detail that only showed up under test: the customer writes "*me arrependi*" and the manual says "*Direito de Arrependimento*". Untreated, section 4.1 was never retrieved. The fix was cutting stems at 5 characters — a poor man's stemmer, but enough for this corpus and without the false positives of a full one. With it, all 11 test questions bring the right section into the top 3.

PDF extraction

pypdf's default mode returns this manual with one word per line, which destroys any chance of detecting headings. `layout` mode preserves the visual structure, and then the 29 numbered sections fall out of a one-line regex.

Prompt: identity in the prompt, facts in the tools

The system prompt holds persona, tone, routing rules and limits — and **no store data**. Address, opening hours and accepted payment methods are not written there, stable as they are. If they were, there would be two sources of truth for the same fact, and they would diverge the first time the manual was updated.

The tone comes from section 7.1 of the manual: "*informal but professional (...) as if talking to a friend who knows music*". The agent is called Dani.

Assumptions made

The agent's reference date is configurable, and the default is not today. The data runs from 2025-10-15 to 2026-03-22. Running the agent on the real system date would put every order outside the 7-day (withdrawal) and 30-day (defect exchange) windows, making those rules impossible to demonstrate. The default is `2026-03-23`, the day after the last order — the only value that keeps the whole dataset coherently in the past. Configurable through `EMPORIO_DATA_REFERENCIA`; `hoje` uses the real date.

The agent does not use a customer table. `orders.customer_id` points at a table this project does not consume, so the agent does not know who it is talking to and cannot find an order by name, tax ID or email. Identification is by order number, and it does not hand over order data unless the customer supplies that number.

That carries two costs, which I would rather state than hide. The manual asks, in flow 7.2, to "greet the customer by name (if available)" — and the agent never greets. And the customer's city, which decides whether delivery is in the metro area, is out of its reach: `payment_options` takes that as a parameter and assumes Campo Grande by default (see Limitations).

The delivery date is approximated by the delivery estimate. The dataset does not record when the product actually arrived, only `estimated_delivery`. That is what the agent counts the 7 days from, and it states the date it based itself on when answering.

The manual and the catalog contradict each other, and the catalog wins. The manual claims "*more than 300 instruments*" and lists wind and orchestral strings; the CSV has 65 products and nothing in those categories. Availability and price always come from the table.

An expired promotion is treated as non-existent. Of the 25 promotions on file, 4 have `is_active = 1`. The other 21 never reach the agent — it cannot offer them even by accident. Section 7.3 of the manual requires checking validity and never promising a lapsed discount; rather than trusting the model with that, the data simply does not exist. (Example 3 shows a customer asking about one of them.)

The manual carries two different phone numbers, and both get used. Section 1.2 gives (67) 3341-4444 as "phone / WhatsApp"; section 7 gives (67) 3321-4500 as "the WhatsApp channel". In the example conversations the agent cites one or the other depending on which section it retrieved. I did not invent a tie-break: both are in the document the store provided, and picking one would be me deciding a business fact. This is the kind of thing you confirm with the client before going to production.

The agent informs, it does not act. It does not cancel orders, issue refunds, apply discounts or reserve products — the CSVs are the source of truth and writing to them from a conversation would require authenticating the customer, which we cannot do.

Known limitations

There is no automated evaluation of the agent. The 27 tests cover the tools, which are pure functions — they lock down the manual's rules (PIX not stacking with a promotion, minimum instalments, deadline windows) without spending an LLM call. What is *not* covered is the model's decision: nothing guarantees it will call `search_policies` on the right question. This is the most serious gap in the project.

Product search requires every word to appear. A search for "violão elétrico nylon" filters on each term; if the customer misspells the model, there is no similarity fallback.

BM25 does not understand synonyms. "Posso pagar fiado?" does not retrieve the payment section, because no word matches. Stem cutting solves inflection, not vocabulary.

A two-topic question drowns one of them. This showed up in the first round of examples: the agent searched "*pedido atrasado não entregue, contato da loja e WhatsApp*" in a single call, the delivery terms dominated the ranking and the contact section did not come back even in the top 6 — so the agent told the customer it did not have the number. I mitigated it in the prompt, asking for one search per topic, and the regenerated conversations now make short, focused calls. That is mitigation, not a fix: the fragility belongs to the retriever, and hybrid search removes it.

Shipping assumes Campo Grande. `payment_options` defaults to `metro_area_delivery=True`, and the agent has no way to know the customer's city — it would have to ask, and it does not always ask. For someone outside the metro area the correct rule is per-postcode quoting (PAC, SEDEX or Jadlog), not the "free shipping above R$500" metro rule. This is the limitation that bothers me most of those left, because it produces a *wrong* answer rather than an incomplete one. The real fix is not prompting, it is having the customer's city alongside the order.

History grows without bound. A long conversation resends everything each turn. With a 1M context window and short support exchanges this is not a practical problem, but there is no compaction.

One conversation at a time. The synchronous `SqliteSaver` is fine for a CLI; an API with concurrency would want the async version.

With more time

The first thing would be the **evaluation set**: 30 to 40 questions labelled with the tool that *should* be called and a fact that *must* appear in the answer. Without it, any change to the prompt is guesswork — there is no way to know whether it helped or hurt. That is what separates a prototype from something maintainable.

After that:

- **Hybrid policy search** — BM25 combined with embeddings would solve the synonym problem, which is the ceiling of the current approach.
- **Prompt caching** on the system block and tool definitions. The prefix is already stable by construction, so it is nearly free.
- **A customer table** and, with it, greeting by name, the city for shipping calculations, and purchase history for recommendations.
- **Human handoff** — an explicit trigger when a customer complains or asks for a person. Today the agent points them to the store but records nothing.
- **An API** (FastAPI) instead of the CLI, which is what a real WhatsApp integration would need.

Example interactions

In `examples/`, five conversations produced by `scripts/generate_examples.py` running against the real model — including which tools the agent chose to call on each turn. None were written by hand. The conversations themselves are in Portuguese, since that is the language the agent works in.

#	Scenario	What it shows
1	Catalog by price range	Catalog query with a numeric filter
2	Store info and scope	Policies + turning down accessories
3	Expired promotion	**Non-trivial** — the customer claims a discount that no longer applies
4	Order return	**Non-trivial** — crosses order data with the policy's deadline window
5	Out of scope and tracking	Polite refusal + two orders in different states

Use of coding assistants

I used **Claude Code** (Opus 5) in an interactive session, start to finish. The workflow was deliberately in two phases.

Exploration first, no code. I asked it to read the brief and the data and bring me what was actually there before proposing any architecture. That pass produced the findings that shaped the project most: that only 4 of the 25 promotions are live — and that the Takamine GD20, the product suggested in the brief itself, carries an expired one, which became example 3. The mismatch between the real date and the end of the data also surfaced here, before it could become a bug.

Then implementation in verifiable blocks. Data → policies → tools → graph → CLI, with tests running at each step and one commit per block. The commits in the repository follow the order in which the blocks were built and validated.

The **architectural decisions were mine**, taken over options I asked to see with their trade-offs: function calling over a SQL agent, LangGraph over a hand-rolled loop, BM25 over embeddings, a configurable reference date over rewriting the CSVs. The justification for each is above — and the point of using the assistant was to reach them with the data in hand, not in the abstract.

Where it paid off most: sweeping the CSVs for edge cases (discontinued items still in stock, orders without tracking, duplicate promotions on the same product) and turning those into tests. Where I had to correct it: the first version of the policy index swallowed parent sections with no body of their own (`5.`, `6.`, `8.`) and failed to retrieve section 4.1 for "*me arrependi*" — both only surfaced because retrieval was measured instead of assumed to work.

Appendix — example conversations

Full transcripts of the sessions cited in *Example interactions*, generated by `scripts/generate_examples.py`. The conversations are in Portuguese — see *Language convention*.

Catálogo com filtro de preço

O cliente dá uma faixa de preço e o agente consulta o catálogo em vez de responder de memória. O filtro roda sobre o preço que o cliente de fato paga, não sobre o de tabela.

> Gerado por `scripts/generate_examples.py` · modelo `claude-haiku-5-5` · data de referência do agente: 2026-03-23

---

cliente: oi! quais opções de violões disponíveis custando até R$1000?

🔧 search_products(category='Violões', max_price=1000) · active_promotions()

Dani: Tenho 8 violões disponíveis até R$ 1.000. Os preços são no cartão, e o PIX dá 5% de desconto:

- Tagima Memphis AC-39 Nylon Natural: R$ 429,90 (PIX R$ 408,40), 15 em estoque
- Rozini RC-104 Nylon Estudante: R$ 459,90 (PIX R$ 436,90), 10 em estoque
- Shelby SN-100 Folk Aço Preto: R$ 549,00 (PIX R$ 521,55), 5 em estoque
- Giannini GF-1R Aço Folk Sunburst: R$ 579,90 (PIX R$ 550,90), 7 em estoque
- Yamaha C40 Nylon Natural: R$ 599,90 (PIX R$ 569,90), 12 em estoque
- Giannini GN-15 Nylon Cedro Natural: R$ 689,00 (PIX R$ 654,55), 6 em estoque
- Yamaha F310 Aço Natural: R$ 699,90 (PIX R$ 664,90), 14 em estoque
- Tagima Dallas Tuner Aço Natural: R$ 749,00 (PIX R$ 711,55), 9 em estoque

Não há promoção ativa nessa faixa de preço para violões. Se você quiser cordas de aço ou de nylon, é só falar que eu filtro. Também posso detalhar algum modelo específico. Ficou faltando alguma coisa?

cliente: o Yamaha F310 é bom pra quem tá começando?

🔧 product_details(product_id=87)

Dani: Pelo que a ficha traz, sim, ele é uma escolha bem comum pra quem está começando. A descrição diz que é um dos modelos mais vendidos da Yamaha para iniciantes e intermediários, com corpo robusto e timbre encorpado.

Algumas especificações: tampo de Spruce (picea), laterais e fundo de Meranti, braço de Nato, cordas de aço, escala de 634 mm e sem captação elétrica. Ele é só acústico, então serve para tocar em casa ou em roda, mas não liga em amplificador.

Preço: R$ 699,90, ou R$ 664,90 no PIX. Tem 14 unidades em estoque.

Se você quer algo mais leve no toque e mais barato, o Yamaha C40 de nylon (R$ 599,90) é outra opção comum para começar. As cordas de nylon são mais macias nos dedos, mas o som é diferente do aço. Ficou alguma dúvida?

cliente: quanto fica no pix?

Dani: No PIX, o Yamaha F310 sai por R$ 664,90. É o preço de R$ 699,90 com o desconto de 5%. Ele não está em promoção, então o desconto do PIX vale.

Ficou faltando alguma coisa?

Informações da loja e pedido fora do catálogo

Endereço e horário saem do manual de políticas. Em seguida o cliente pede acessórios, que a loja não comercializa — o agente redireciona sem inventar um produto.

> Gerado por `scripts/generate_examples.py` · modelo `claude-haiku-5-5` · data de referência do agente: 2026-03-23

---

cliente: qual o endereço da loja?

🔧 search_policies(question='endereço da loja')

Dani: A loja fica na Rua 14 de Maio, 3200, Centro, Campo Grande - MS, CEP 79202-333.

Se precisar, o WhatsApp e telefone é (67) 3341-4444. Quer mais alguma coisa?

cliente: vocês abrem sábado que horas?

🔧 search_policies(question='horário de funcionamento sábado')

Dani: No sábado a loja atende das 9h às 13h. Durante a semana, é de segunda a sexta, das 9h às 18h. Domingo e feriados ficam fechados.

Ficou mais alguma dúvida?

cliente: preciso de um jogo de cordas e umas palhetas também, tem?

Dani: Cordas e palhetas são acessórios, e a Empório da Música trabalha só com instrumentos musicais, então infelizmente não temos isso aqui.

Se quiser, posso procurar algum instrumento para você, como violão, guitarra, baixo ou teclado. Tem algo em mente?

Promoção expirada (cenário não trivial)

O Takamine GD20 tem uma promoção de 12% cadastrada com is_active = 0. O cliente pergunta justamente por ela. O agente precisa informar o preço atual e não prometer um desconto que não está mais vigente (manual, seção 7.3).

> Gerado por `scripts/generate_examples.py` · modelo `claude-haiku-5-5` · data de referência do agente: 2026-03-23

---

cliente: quanto custa o Takamine GD20?

🔧 search_products(term='takamine GD20')

Dani: O Takamine GD20 Dreadnought Natural está por R$ 2.199,00. No PIX sai por R$ 2.089,05, com o desconto de 5%.

Está disponível, com 5 unidades em estoque. Quer que eu veja as opções de parcelamento no cartão ou o valor do frete para sua cidade?

cliente: ué, eu vi num anúncio que tinha 12% de desconto nele. ainda vale?

🔧 active_promotions()

Dani: Não encontrei nenhuma promoção de 12% no Takamine GD20 hoje, então esse anúncio deve ser de uma campanha que já encerrou. O preço vigente dele é R$ 2.199,00, e no PIX fica R$ 2.089,05 (5% de desconto). Ele está com 5 unidades em estoque.

Se quiser, posso ver outras opções nessa faixa de preço para comparar.

cliente: e tem alguma promoção rolando hoje então?

Dani: Hoje estão rolando estas promoções:

- Ohana CK-20 Concert Natural (ukulele): de R$ 549,00 por R$ 439,20, com 20% de desconto. Tem 6 em estoque.
- Crafter HT-100 Folk Aço Natural (violão): de R$ 2.399,00 por R$ 1.967,18, com 18% de desconto. Tem 3 em estoque.
- Kalani KAL-700T Tenor Natural (ukulele): de R$ 359,00 por R$ 323,10, com 10% de desconto. Tem 11 em estoque.
- Taylor 110e Dreadnought Natural (violão): de R$ 5.999,00 por R$ 5.519,08, com 8% de desconto. Tem 4 em estoque.

Nessas promoções o desconto de 5% do PIX não se aplica. Quer que eu veja as condições de pagamento de alguma delas?

Devolução com cálculo de prazo (cenário não trivial)

O agente cruza dois mundos: o pedido (data de entrega, vinda do CSV) e a política (janela de 7 dias para arrependimento). O prazo do pedido 7 já venceu, então a resposta certa é negar com transparência — e apontar a garantia legal de 90 dias, que ainda está de pé.

> Gerado por `scripts/generate_examples.py` · modelo `claude-haiku-5-5` · data de referência do agente: 2026-03-23

---

cliente: me arrependi da minha compra, posso devolver meu pedido?

🔧 search_policies(question='devolução por arrependimento prazo')

Dani: Dá para sim, pode devolver. Pela nossa política, em compras online você tem até 7 dias corridos após o recebimento para pedir a devolução, sem precisar justificar. O produto precisa voltar na embalagem original, sem sinais de uso, com todos os acessórios e manuais. O reembolso sai na mesma forma de pagamento usada na compra, em até 10 dias úteis, e o frete da devolução fica por conta da loja.

Para eu conferir o seu pedido e os prazos certinhos, me manda o número do pedido, por favor. Ele aparece na confirmação de compra.

cliente: é o pedido 7

🔧 lookup_order(order_id=7) · search_policies(question='telefone WhatsApp contato da loja')

Dani: Achei o pedido #7 aqui, e infelizmente o arrependimento não vale mais para ele. Ele foi entregue em 17/02, há 34 dias, e o prazo de 7 dias corridos já passou.

Mesmo assim, ele ainda está coberto pela garantia legal de 90 dias contra defeito de fabricação, que vai até cerca de 18/05. Ou seja, se a Shelby SU-21S apresentar algum defeito de fábrica, dá para acionar a troca ou o conserto por aí. Arrependimento sem defeito, porém, não se enquadra.

Se quiser conversar com a equipe sobre a sua situação, o contato é (67) 3341-4444, pelo WhatsApp ou telefone, de segunda a sexta das 9h às 18h e sábado das 9h às 13h. Eles podem olhar o caso com você.

Ficou mais alguma dúvida?

cliente: e se for defeito de fábrica, muda alguma coisa?

🔧 search_policies(question='defeito de fabricação troca garantia')

Dani: Sim, muda bastante, mas o caminho é outro. A troca direta na loja por defeito vale só nos primeiros 30 dias após a compra, e o pedido #7 já passou desse prazo (a compra foi em 03/02).

Depois disso, a garantia legal de 90 dias contados do recebimento (que vai até cerca de 18/05 no seu caso) cobre defeito de fabricação. Nesse período, você aciona a garantia com a gente: a loja pode intermediar o processo com o fabricante a seu pedido. Também vale checar o certificado de garantia que veio com a Shelby, porque muitos fabricantes têm garantia própria, de 6 meses a 2 anos, que pode estar valendo além disso.

Ficam de fora danos por mau uso, quedas, umidade excessiva ou modificações não autorizadas.

Se você estiver com um defeito, o melhor é chamar a equipe no (67) 3341-4444, WhatsApp ou telefone, de seg a sex das 9h às 18h e sábado das 9h às 13h. Eles vão olhar o caso e te dizer os próximos passos. Quer que eu ajude com mais alguma coisa?

Pergunta fora de escopo e rastreio de pedido

Pergunta sem relação com a loja é recusada com leveza. Na sequência, um pedido já despachado e outro ainda sem rastreio — o agente explica a diferença em vez de inventar um código.

> Gerado por `scripts/generate_examples.py` · modelo `claude-haiku-5-5` · data de referência do agente: 2026-03-23

---

cliente: qual a capital da Mongólia?

Dani: Essa eu não consigo ajudar, sou do atendimento da Empório da Música e só cuido de instrumentos e das coisas da loja. Se estiver procurando algum instrumento, violão, guitarra, teclado ou outro, é só me falar!

cliente: haha beleza. cadê meu pedido 8?

🔧 lookup_order(order_id=8) · search_policies(question='telefone WhatsApp contato atendimento')

Dani: Pelo que aparece aqui, o pedido #8 já foi enviado (Kala KA-C Concert Mogno), com rastreio BRJL5544332BR. Mas a previsão de entrega era 28/02, e ele ainda não consta como entregue. Ou seja, está atrasado há cerca de três semanas.

Não consigo ver onde o pacote está agora, então o melhor é a equipe verificar com a transportadora. O canal de WhatsApp de atendimento é o (67) 3321-4500. No cadastro da empresa aparece também o (67) 3341-4444 como telefone/WhatsApp, então, se um não responder, tente o outro. Tenha em mãos o número do pedido e o código de rastreio.

Se o pacote não aparecer, a equipe vai te orientar sobre os próximos passos. Ficou alguma dúvida?

cliente: e o pedido 13, que tá demorando?

🔧 lookup_order(order_id=13)

Dani: O pedido #13 (Ohana BK-35E Barítono Elétrico Natural, R$ 1.899,00, pago via PIX) está aguardando pagamento. Ele ainda não foi despachado, por isso não tem código de rastreio. O rastreio é enviado por e-mail e WhatsApp assim que o pedido sai.

Se você ainda não pagou, a demora é por isso e é preciso concluir o pagamento. Se você já pagou por PIX, fale com a equipe para confirmar o recebimento: (67) 3321-4500 ou (67) 3341-4444. Tenha o número do pedido em mãos.

Ficou alguma dúvida?